

Final Paper CISC 4631

By Kevin Kairouz & Prosad Shaha

PROBLEM

Every day, as humans work to advance the real world, distractions hold us back. Because of this, we hurt our productivity and potentially miss out on what that day or week has to offer. It may be a new invention, algorithm, or idea that solves an unsolved problem. One of the most common productivity killers in the 21st century is social media. What was once looked at with skepticism 20 or so years ago has blossomed into a large part of everyday life for many. However, while technological advancement has brought some benefits worldwide, the hours spent every day on such applications compound over time, time we can never get back. While it is unreasonable and rather unethical to eliminate hours spent on social media fully, it is healthy to limit it and instead prioritize other areas of life to increase overall productivity and health. Areas of life, such as sleep and studying, are crucial to productivity as both are key to self-improvement and cognitive development.

DATA & PREPROCESSING

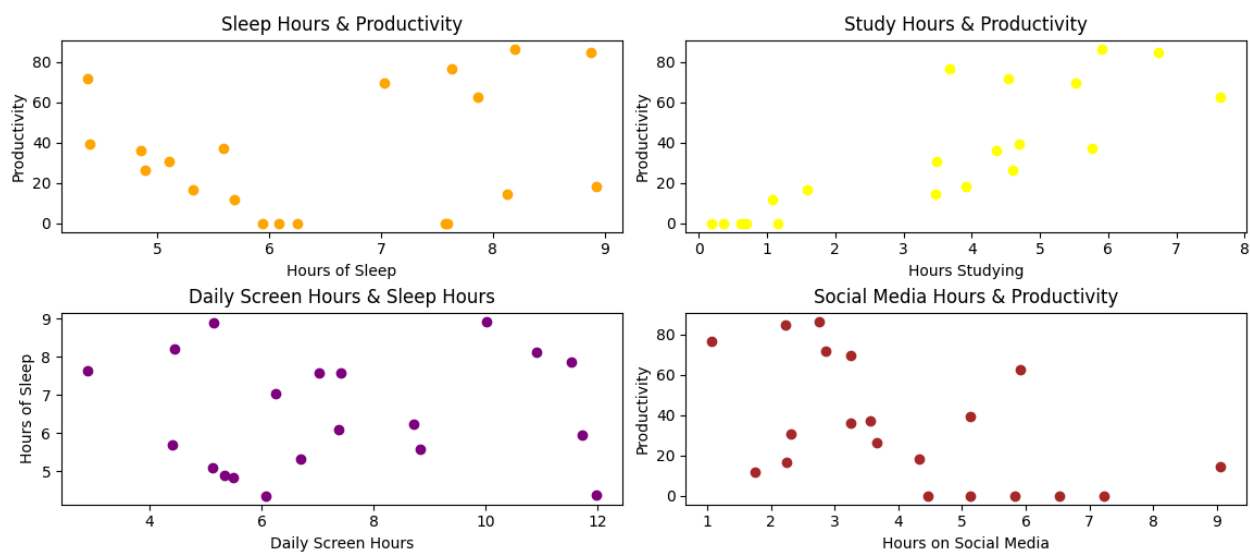
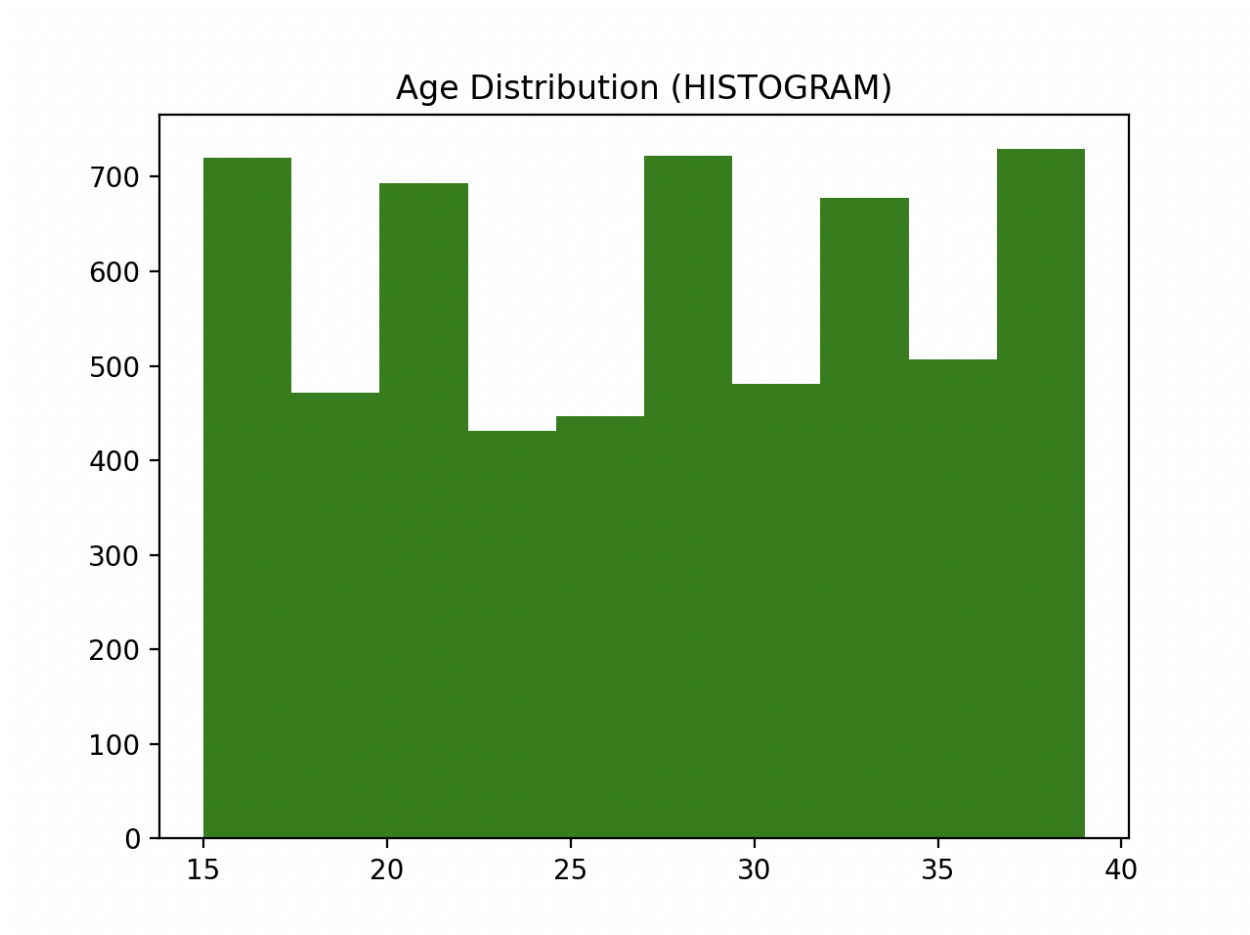
Data comes in many shapes and sizes, and we can never get enough of it. It can be found anywhere, as data can be derived from files such as CSVs, APIs, or even web scraping. For the models proposed in this project, our data was collected via a dataset uploaded to Kaggle. Despite the CSV file appearing clean, we utilized the pandas library

to modify data types and drop extraneous columns. During pre-processing, we specifically used the `pd. to_numeric` function alongside the `drop` function to achieve this. Cleaning the dataframe enabled us to properly split the data into X and Y and train and test our Gradient Boosting Model. For our Neural Network (Deep learning), we used the PyTorch deep learning framework from Meta. Pre-processing for this model, while similar to that for our Gradient Boosting model, had a handful of caveats. This is because, unlike scikit-learn, PyTorch is fundamentally stricter with respect to training and pre-processing, as the input to your neural network must be a tensor (multi-dimensional array) of a single type; a typical pandas dataframe would cause errors due to its leniency. Our tensors were scaled using scikit-learn's `StandardScaler` and then placed into a `TensorDataset` object. The reason is that it allows one object to carry multiple tensors, namely your X and Y training data. Lastly, we pass the object to the `DataLoader` class's constructor. The `DataLoader` object is an iterable that allows us to iterate over batches/each instance of a tensor during neural network training.

During an Exploratory Data Analysis, many meaningful insights can be found in the data. In the beginning, it is important to familiarize ourselves with the data we are working with, including its diversity. A good way to do this is by utilizing a histogram. In our case, we wanted to observe the most common age ranges. Based on our histogram below, the most common age ranges are 15 - 18, 26 - 28, and 35 - 39.

After looking at the data more broadly, we extracted what we believed to be key features in our X data and measured how much they affected our Y, Productivity score. In the featured subplot, we can draw a strong conclusion that moderate to high levels of

studying, greater sleep, and less time spent on social media are directly correlated with higher productivity.

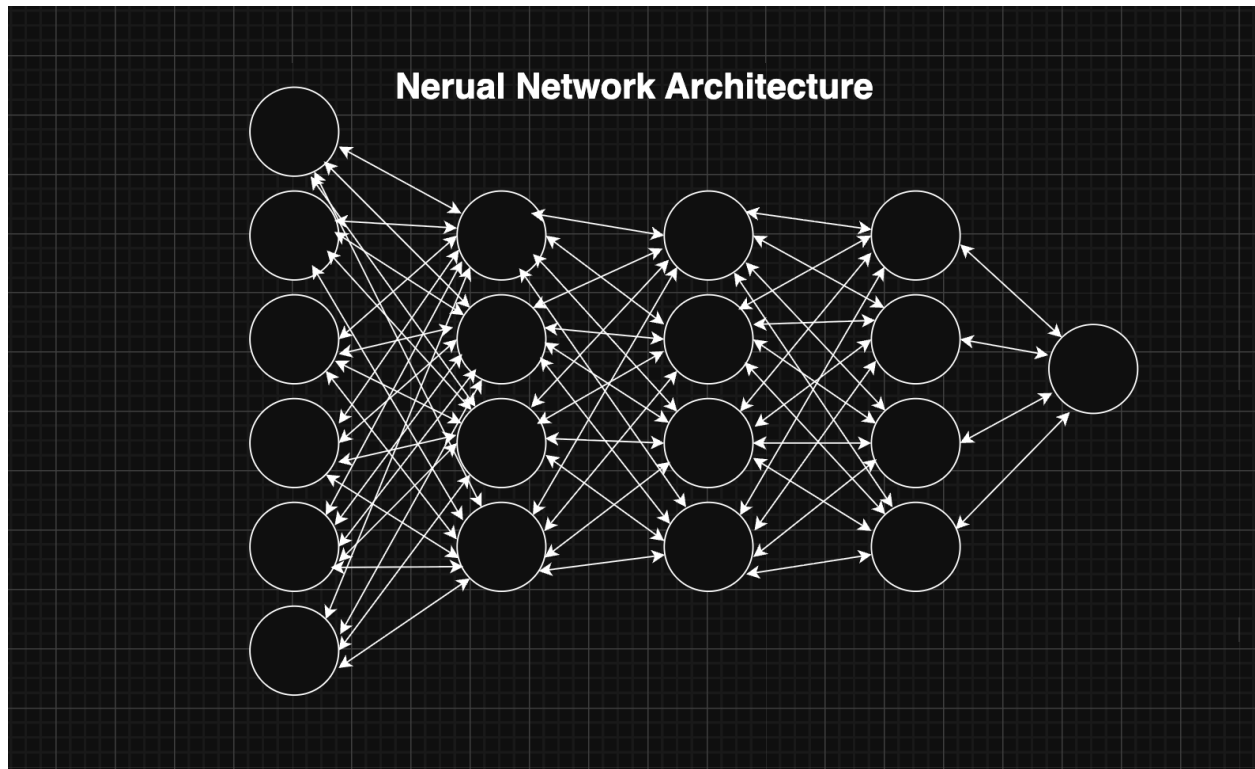


METHODS

As mentioned earlier, our 2 models were built using scikit-learn (Gradient Boosting Regressor) and PyTorch (Neural Network). Both models were constructed in different ways, as each library employs a different approach to model assembly and composition. In sklearn, we experimented with different models, those being Linear Regression and Random Forest. Linear Regression is widely used in Machine Learning for its consistent reliability and ease of use, as it actively tries to find the line of best fit. Random Forest is an ensemble approach, known for better performance at identifying relationships and patterns in complex datasets. Gradient Boosting builds on Random Forest, as it is an optimized version of the algorithm. With Random Forest, each tree is built randomly; therefore, each tree is different and does not know how the other trees are designed. However, with Gradient Boosting, each tree is built sequentially and learns from its predecessor, with the amount of learning from its predecessor contingent on the learning rate defined in the constructor. Each tree in Gradient Boosting tries to correct its successor's tree. The performance of this algorithm is heavily affected by the following parameters: `n_estimators` and the learning rate (known as `lr` in the class). When the learning rate is high, `n_estimators` must be low, as you risk overfitting: the tree will fail to generalize and find patterns. Conversely, when the learning rate is low, and `n_estimators` are likewise, you risk underfitting, as you do not provide enough data for your model to generalize, and the amount it learns is insignificant. Using HyperParameter tuning techniques that the scikit-learn library offers, such as `GridSearchCV`, we were able to identify the most efficient parameters for the model. The parameters are 150 decision trees (`n_estimators`) and a learning rate of 0.1. These two

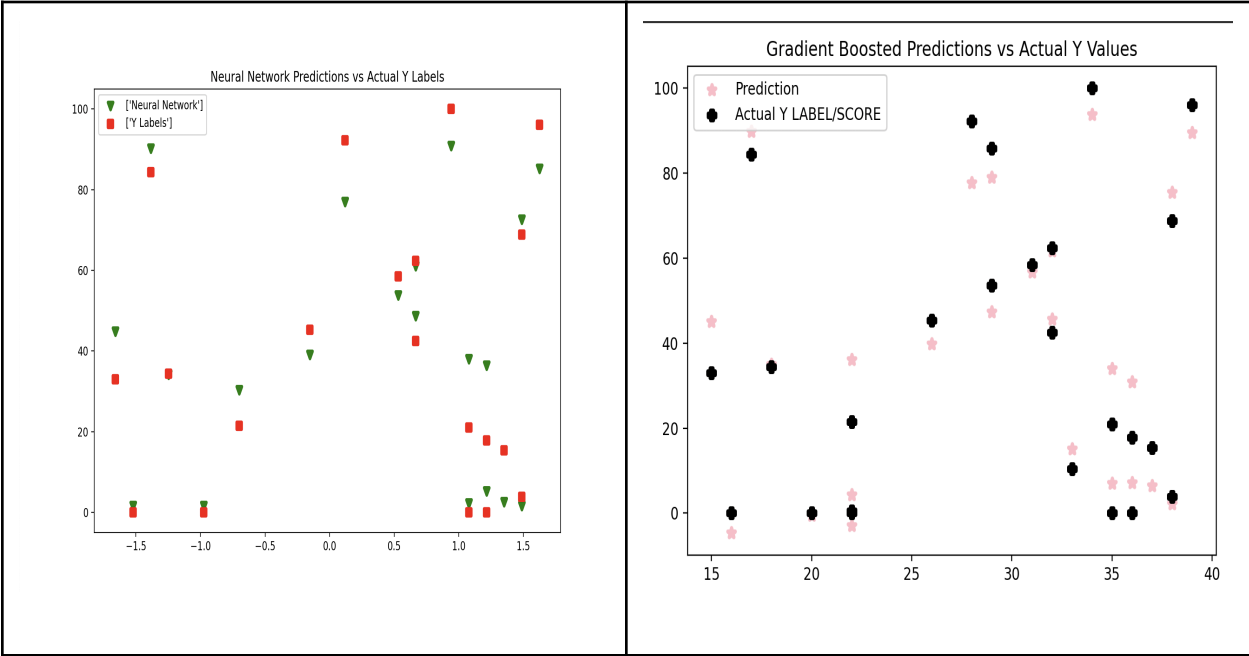
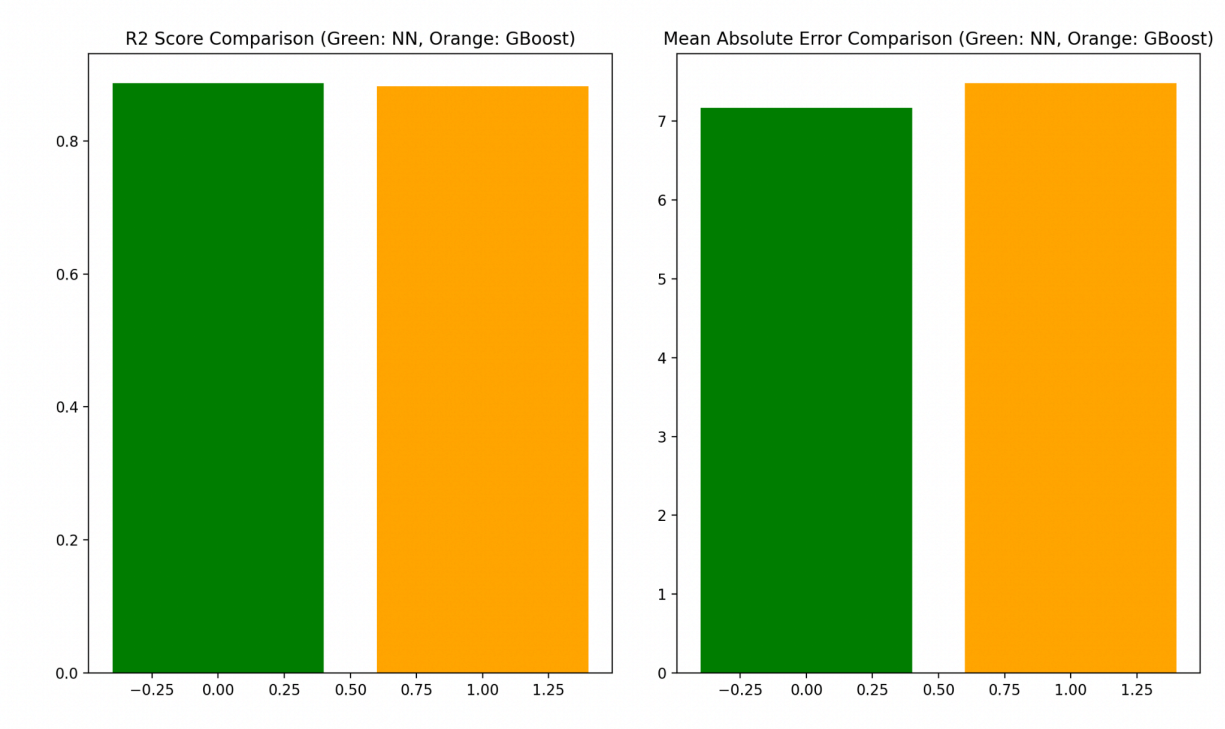
provide a slow-and-steady approach: with each tree, it learns a little bit from its predecessor, so we neither overfit nor underfit but instead tune the model correctly.

On the other hand, PyTorch differs significantly from scikit-learn in terms of architecture and training. The Neural Network, despite being computationally expensive, offers greater control and flexibility, enabling further tuning and modification of the model. There are many different types of Artificial Neural Networks, such as Convolutional Neural Networks, Recurrent Neural Networks, and LSTM. For the sake of this project, we collectively decided that a Linear Neural Network, also known as a Fully Connected or Feed-Forward Neural Network, was sufficient due to its flexible, simple architecture. Knowing that our attribute set has six attributes, our input layer consists of six nodes that connect to our first hidden layer of four nodes. Between each layer lies an activation function, which introduces non-linearity and determines which neuron in the network fires as the network aims to put the pieces of the puzzle together. The activation function used in our Neural Network is ReLU (Rectified Linear Unit), as it offers many advantages in Deep Learning, its biggest being its computational efficiency, which allows faster convergence in deep learning models. An optimizer and a loss function are both needed for our model, as Neural Networks depend on them to backpropagate, compute a penalty, and adjust weights accordingly. Thus, we chose Mean Squared Loss as our loss function and Stochastic Gradient Descent as our optimizer. Our Neural Network has one input layer, three hidden layers, and one output layer. Below is a blueprint of the referenced neural network architecture. From here, we set the number of epochs to 100, since our dataset was somewhat complex and our neural network had only a few hidden layers.



MODEL ANALYSIS & Evaluation

After training each model, we were able to directly compare predictions and metrics. The two metrics we used were Mean Absolute Error (MAE) and R2 Score (R^2). Mean Absolute Error is defined as the average absolute difference between predicted and actual values. R2 score, on the other hand, is a measure of how well the model/line fits the data. Below are two bar charts that showcase the metrics for each model. The Neural Network, though computationally expensive and more complex, achieves a lower MAE and fits the data better, attaining roughly a 7.2 MAE and an 88.4 R2 score, while our Gradient Boosting model attains an 8 for its MAE and an 88.2 for its R2 score, slightly underperforming its counterpart.



CONCLUSION

In an ongoing world of events, it is important as humans to improve one day at a time. As mentioned earlier, it is unethical and unreasonable to expect individuals to cut off social media and screen time completely. However, limiting it is possible, and in the long term, many may see improvements in overall productivity, including energy levels, cognitive ability, and concentration. Our project has a full-stack aspect, as we developed it into a full-stack web application. This allows users to calculate their productivity score, see how well they are doing on their journey, and compare themselves to the prior week, month, or even year! Having a real-world impact and helping many slowly become the best version of themselves.